

FORMATION EMBARQUÉE

TP2 — Seance 1

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 2 — Fiche de séance 1 : spécification et émetteur UART (2 h)

En-tête pédagogique

Objectifs	Écrire une spécification chiffrée avant le code ; calculer une erreur de baudrate ; coder et simuler un émetteur UART ; mesurer au chronogramme
Prérequis	Module 04 en entier ; TD 04 exercices 1-2 (mux, compteur) réussis en simulation
Outils	GHDL + GTKWave installés (<code>sudo apt install ghdl gtkwave</code>) ou edaplayground.com
Livrable	<code>uart_tx.vhd</code> + testbench, durée de bit vérifiée au curseur : 434 cycles pile

Déroulé minuté

0:00-0:10 — Préparer l'atelier

Crée un dossier de travail avec le script de simulation :

```
mkdir tp2-uart && cd tp2-uart && git init
```

```
#!/bin/sh
# simule.sh <sources...> <testbench.vhd> - analyse, élabore, simule, affiche
set -e
ghdl -a --std=08 "$@"
TB=$(basename "${@: -1}" .vhd)
ghdl -e --std=08 "$TB"
ghdl -r --std=08 "$TB" --wave="$TB.ghw" --stop-time=5ms
gtkwave "$TB.ghw" &
```

`chmod +x simule.sh` . Vérifie l'installation avec le mux du TD 04 : `./simule.sh mux4.vhd tb_mux4.vhd` doit ouvrir GTKWave.

0:10-0:40 — La spécification (papier, PAS de code)

Remplis ce tableau dans un fichier `SPEC.md` — c'est un livrable noté :

Paramètre	Valeur	Justification
Format	8N1	1 start + 8 données (LSB d'abord) + 1 stop
Baudrate	115 200	standard de debug
F horloge	50 MHz	générique, adaptable
Ticks par bit	$\lfloor 50\,000\,000 / 115\,200 \rfloor = \mathbf{434}$	division entière
Baud réel	$50\,000\,000 / 434 \approx \mathbf{115\,207}$	
Erreur par bit	$(115\,207 - 115\,200) / 115\,200 \approx \mathbf{+0,006\%}$	
Dérive sur 10 bits	$\approx 0,06\%$ de la durée de trame	$\ll \frac{1}{2}$ bit (5 %) : OK

Les calculs à savoir refaire de tête en entretien : 1. **Ticks par bit** = $f_{\text{clk}} / \text{baud}$ (division entière → erreur d'arrondi). 2. **La limite** : le récepteur échantillonne au milieu du bit ; l'erreur cumulée au 10^e bit doit rester $< \frac{1}{2}$ bit, soit $\sim 5\%$ par bit → un couple horloge/baud qui donne $> \sim 2\%$ d'erreur d'arrondi est à proscrire. 3. Contre-exemple à calculer : $1\text{ MHz} / 115\,200 = 8,68$ → arrondi à 8 ou 9, erreur ≈ 8 ou 4% → **hors spec**. C'est pour ça que les vieux quartz « UART-friendly » font $1,8432\text{ MHz}$ ($= 16 \times 115\,200$).

Dessine aussi la trame 8N1 pour l'octet 0x55 (repos haut, start bas, **1,0,1,0,1,0,1,0** LSB d'abord, stop haut) — tu la compareras au chronogramme à 1:50.

0:40-1:30 — Coder l'émetteur

Écris `uart_tx.vhd` **de mémoire** en t'appuyant sur la FSM 4 états (REPOS → START → DONNEES → STOP). Ne regarde le corrigé (`code/vhdl/uart_tx.vhd`) qu'en cas de blocage > 15 min. Les 4 pièges qui te guettent :

1. **Capturer `tx_data` dans un registre interne** à l'acceptation du `tx_start` — sinon l'appelant peut changer la donnée en pleine émission.
2. LSB d'abord : `tx <= registre(idx_bit)` avec `idx_bit` qui monte de 0 à 7.
3. Le compteur de ticks se compare à `TICKS_PAR_BIT - 1` (le « off by one » classique : compter de 0 à N-1 fait N cycles).
4. `busy` levé de START à STOP inclus.

1:30-1:50 — Le testbench

Reprends la structure du TD 04 exercice 5 : horloge 50 MHz (`clk <= not clk after 10 ns;`), impulsion `tx_start` d'un cycle, envoi de `x"55"`, attente de `busy = '0'`.

```
./simule.sh uart_tx.vhd tb_uart_tx.vhd
```

1:50-2:00 — Mesure au chronogramme (✓ point de contrôle)

Dans GTKWave : ajoute `tx`, `busy`, `etat`, `cpt_tick`. Avec les **deux curseurs** (clic + clic-milieu), mesure : - durée du bit de start : attendu **8 680 ns** ($434 \times 20\text{ ns}$) pile ; - la séquence des bits de données pour 0x55 : **1,0,1,0,1,0,1,0** — compare avec ton dessin de 0:40 ; - durée totale de trame : $10 \times 8\,680 = \mathbf{86,8\ \mu s}$.

Un écart d'un cycle (8 660 ou 8 700 ns) = piège n°3 ci-dessus : corrige avant de continuer, le récepteur de la séance 2 ne pardonnera pas.

Commit : `git add . && git commit -m "TP2 seance 1 : spec + uart_tx valide"`.

Erreurs fréquentes

Symptôme	Cause	Remède
GTKWave vide	simulation stoppée avant les stimuli (<code>--stop-time</code> trop court) ou pas de <code>wait</code> final	vérifier la console GHDL
<code>tx</code> reste à 'U'	<code>tx</code> affecté seulement dans certains états au lieu de tous	l'affecter dans chaque branche du case
Trame trop courte d'un bit	transition d'état ET incrément de compteur le même cycle mal ordonnés	revoir : remise à zéro du compteur à chaque changement d'état
Le 2 ^e envoi émet l'ancienne donnée	<code>tx_data</code> lu pendant DONNEES au lieu d'être capturé	piège n°1
<code>ghdl: cannot find entity</code>	ordre d'analyse : les sources AVANT le testbench	<code>./simule.sh sources... tb.vhd</code>

Travail à la maison (30 min)

Ajoute un generic `PARITE : boolean := false` qui, à vrai, insère un bit de parité paire entre le bit 7 et le stop (XOR de tous les bits de données — en VHDL : `xor_reduce` ou une boucle). Testbench : vérifie la trame de 0x55 (parité 0) et de 0x54 (parité 1).

➡ Fiche suivante : [Séance 2 — Le récepteur](#)